Application Containers and Docker for Embedded

Advanced Embedded Software Development

with **Dan Walkes**



University of Colorado **Boulder**

Learning objectives:

Understand the difference between containerization and virtualization.

Understand Linux kernel capabilities supporting containerization.

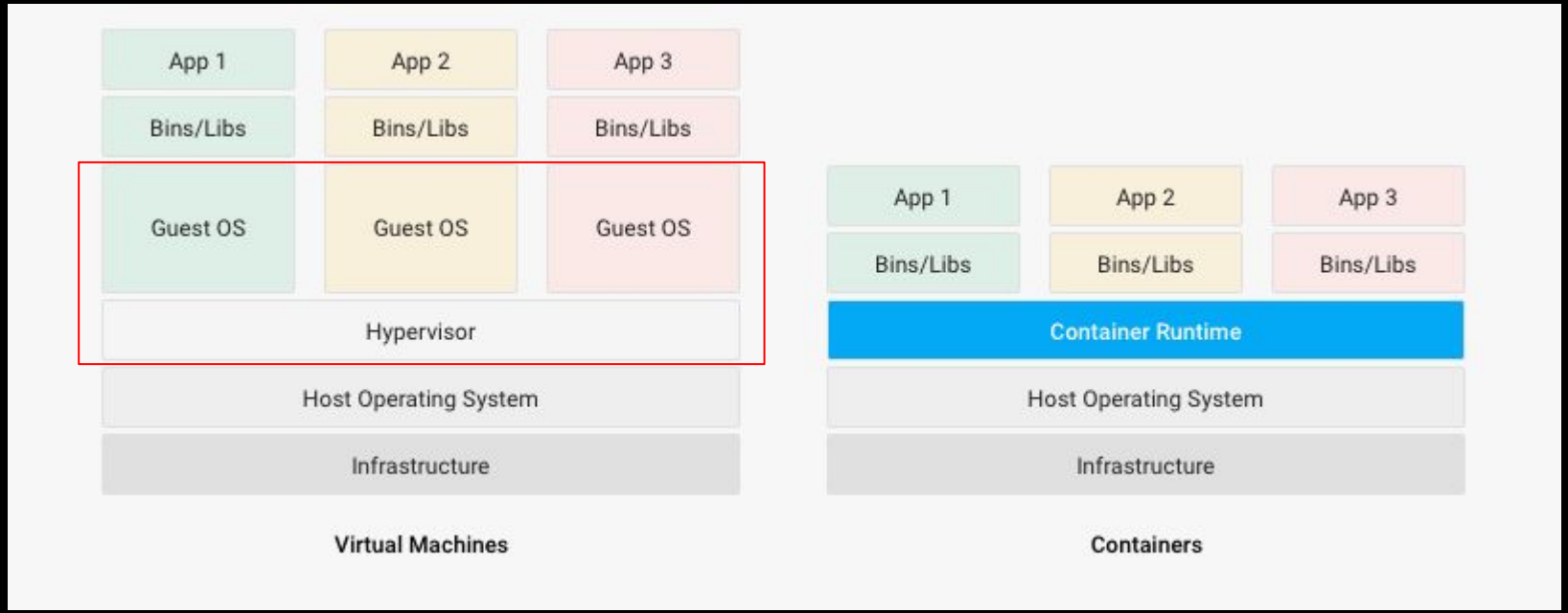
Understand how containers can be used in an Embedded Development workflow.

Application Containers

- Logical packaging mechanism to abstract applications from the environment in which they run
  - Deploy applications consistently regardless of target environment
    - Datacenter, cloud, developer laptop, embedded device
- Developers focus on application logic and dependencies
  - Which applications/libraries need to be installed in the root filesystem?
  - What versions have been tested successfully with my application?
- IT teams focus on deployment and management
  - When should the application be started/stopped?
  - How much memory/storage should I allocate?
  - What ports should it use on the host?

https://cloud.google.com/containers/

Application Containers vs Virtualization



- A container is a VM without a dedicated kernel (shares with the host)

https://cloud.google.com/containers/
https://www.freecodecamp.org/news/comprehensive-introductory-guide-to-docker-vms-and-containers-4e42a13ee103/

Container Kernel Features

- Namespaces
  - Creates virtual isolated instances of a global resource using the “clone” system call (similar to fork).
    - networking
    - ports
    - mount points
    - process IDs
- Control Groups
  - Enforces limits and constraints on groups of processes (memory, CPU, etc).
- Union Mounts
  - Combining multiple directories into a single one which appears to contain combined contents.

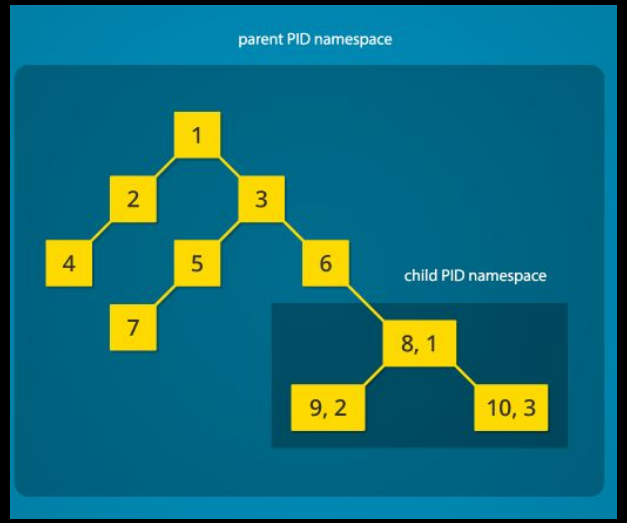
http://man7.org/linux/man-pages/man2/clone.2.html
http://man7.org/linux/man-pages/man7/namespaces.7.html
http://man7.org/linux/man-pages/man7/cgroups.7.html
https://en.wikipedia.org/wiki/Union_mount

Process Namespaces

- Process tree typically contains reference to every process in the kernel.
  - A process with sufficient privileges can attach to another process or kill it.
- Namespaces introduce “nested” process trees
  - Processes belonging to a process tree cannot inspect or kill processes in other trees
- Use clone with CLONE_NEWPID

```
int clone(int (*fn)(void *), void *child_stack,
          int flags, void *arg, ...

CLONE_NEWPID (since Linux 2.6.24)
       If CLONE_NEWPID is set, then create the process in a new PID
       namespace.  If this flag is not set, then (as with fork(2))
       the process is created in the same PID namespace as the
       calling process.  This flag is intended for the implementation
       of containers.
```



http://man7.org/linux/man-pages/man2/clone.2.html
https://www.toptal.com/linux/separation-anxiety-isolating-your-system-with-linux-namespaces

Network Namespaces

- Processes can be created with different set of networking interfaces
  - Network ports on the parent processes may not be accessible on the child process
  - Different loopback interfaces for parent and child networks
- Can create virtual routing between networks from the parent namespace
- Use clone with CLONE_NEWNET when creating the process

```
int clone(int (*fn)(void *), void *child_stack,
          int flags, void *arg, ...
```

CLONE_NEWNET (since Linux 2.6.24)
(The implementation of this flag was completed only by about kernel version 2.6.29.)

If **CLONE_NEWNET** is set, then create the process in a new network namespace. If this flag is not set, then (as with fork(2)) the process is created in the same network namespace as the calling process. This flag is intended for the implementation of containers.

```
Original `net` Namespace:
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group
default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp4s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP mode
DEFAULT group default qlen 1000
    link/ether 00:24:8c:a1:ac:e7 brd ff:ff:ff:ff:ff:ff


New `net` Namespace:
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN mode DEFAULT group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
```

http://man7.org/linux/man-pages/man2/clone.2.html
https://www.toptal.com/linux/separation-anxiety-isolating-your-system-with-linux-namespaces

Control Groups

- Organize processes into hierarchical groups
- Limit resources in that group
- Uses the cgroupfs pseudo-filesystem to control
  - Limit the amount of CPU time and memory available to a group
  - Freezing and resuming execution of processes in a group

Create a new cgroup named *groupname*:

```
# mkdir /sys/fs/cgroup/memory/groupname
```

Example: set the maximum memory limit to 100MB:

```
# echo 100000000 > /sys/fs/cgroup/memory/groupname/memory.limit_in_bytes
```

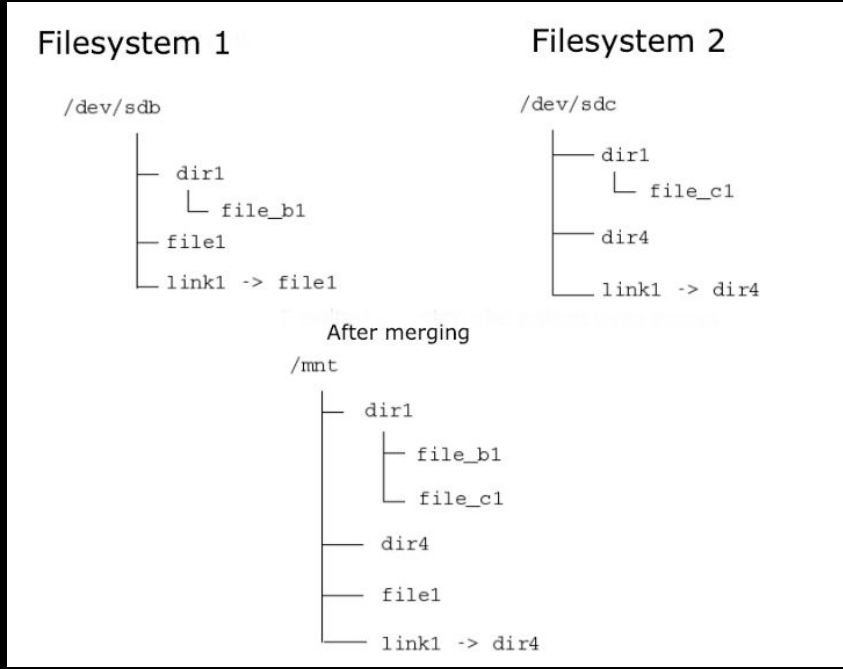
Move a process to the cgroup (note: only one PID can be written at a time, repeat this for each process that must be moved):

```
# echo pid > /sys/fs/cgroup/memory/groupname/cgroup.procs
```

http://man7.org/linux/man-pages/man7/cgroups.7.html
https://wiki.archlinux.org/index.php/cgroups

Union Mounts

- Started in the early 90s to support writeable CDs
- Three implementations since:
  - UnionFS
  - aufs
  - OverlayFS - First version accepted into the mainline kernel
- OverlayFS:
  - Sets up a hierarchy between filesystems
  - Allows a read/write overlay on top of a read only overlay, with read/write portion in RAM

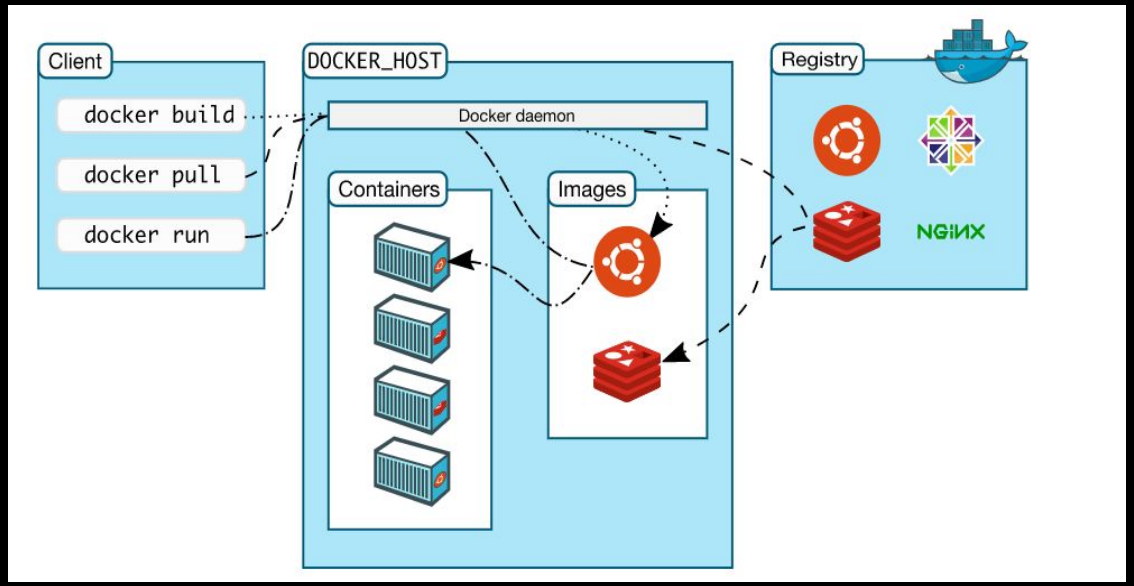


https://lwn.net/Articles/396020/
https://www.freecodecamp.org/news/comprehensive-introductory-guide-to-docker-vms-and-containers-4e42a13ee103/
https://www.kernel.org/doc/Documentation/filesystems/overlayfs.txt

Docker

- Open platform for containerized applications
- Based on a Docker Engine client-server application

- docker daemon
  - Starts/runs/controls containers
- docker client
  - Interacts with docker daemon to perform commands
- docker registry
  - Stores docker images (Docker Hub)



https://docs.docker.com/engine/docker-overview/#the-underlying-technology

Docker Setup - Ubuntu Host

- See https://docs.docker.com/install/linux/docker-ce/ubuntu/ to run an existing container
  - Setup apt sources to include docker CE
  - Install docker-ce and docker CLI
  - sudo docker run hello-world
    - Finds hello-world on docker hub https://hub.docker.com/_/hello-world/
    - Starts a multi-stage build step using multi-stage builds feature
    - Starts container to build based on Dockerfile.build
      - Pulls down debian:stretch-slim container
      - Runs commands to setup compile environment and cross compile for possible target platforms
  - Compiles hello.c and copies to root of container and runs inside

https://docs.docker.com/install/linux/docker-ce/ubuntu/
https://docs.docker.com/develop/develop-images/multistage-build/
https://github.com/docker-library/hello-world/blob/master/Dockerfile.build

Docker Setup - Buildroot

- Add docker components to your image
  - docker-engine
    - Requires changing toolchain from rpi default to glibc
  - docker-cli
- Increase default rootfs size
  - 128M default for RPI is too small for docker components (500M is sufficient)
- Boot new sdcard image on target
- Use check-config.sh script to find necessary kernel packages for docker support (including cgroup, overlay) and add to build
- Build with new kernel and test on hardware.
- See https://github.com/cu-ecen-5013/buildroot-assignments-base/wiki/Docker-on-Buildroot for overview of steps and completed pull request.

https://github.com/docker/engine/blob/master/contrib/check-config.sh
http://buildroot-busybox.2317881.n4.nabble.com/PATCH-support-testing-add-docker-docker-compose-tests-td215426.html#a215544

Container Use for Embedded

- When might you use containers/docker for Embedded Development?
- Development Host System(s):
  - Creating a consistent build/development environment
  - Creating automated test frameworks
  - Deploying test frameworks to Continuous Integration/Continuous Deployment automated test platforms.
- Target System:
  - Pulling in dependencies quickly for test purposes
    - Run Ubuntu on Buildroot (sudo apt-get install)
  - Supporting optional/conflicting product configurations
    - Bring in open source component X that requires specific versions of python/libraries.
    - Add a second web server/application redirected to a port other than :80